

What should I draw?



CS 340



Bitwise Operations 0b01

clicker.cs.illinois.edu

Q1

~Code~
340



Updates

1. Exam 1 scores are released.
 - a. More information on Campuswire

retake
thursday

2. MP2 - PNG due today

PMPP

3. MP3 - out today (due next Tuesday)

MP3.1 - utf8

MP3.2 - bit sets

MPP
Thursday after class
SC3403 3:30-5:00

4. HW5 - out today (due a week from Wednesday)

Pro Tips

- Check the lecture notes on the google slides for a typed version of what I write
- Post a CBTF exam in the grainger library, check out the IDEA lab for VR, puzzles, and video games.

<https://www.library.illinois.edu/enx/idea-lab-homepage/>

Bitwise Operations

Today's LGs - Build on your mental model of how data is stored and interpreted on a computer (1's and 0's).

Be able to converse about bits

Be able to shift bits

Be able to apply logical operations at a bit level

Be able to use bit operations to isolate the bits you need

Be able to convert between code points and UTF-8

Agenda

1. Addressing misconceptions
 - a. Important terms
 - b. Representing bits versus interpreting bits
2. Bitwise Operations 
 - a. Bit shifting
 - b. Bit logic operations
2. Bit Mask
3. MP~~3~~ - UTF-8

Important Terms 0b01

Bit Vector - fixed length sequence of bits
in C, char $\rightarrow$ 8-bit vector



Most/least significant bit

Clear - verb to replace 1 bit with 0, replace all bits with 0

Important Terms Ob10

ith bit



Set — set this bit (make it a 1)

The 2nd bit is set

set-collection of things no repeats

Zero

→ 0000 0000

Representing Bits Versus Interpreting Bits

- Information is stored as.... bits (1, 0)

100100100 → 0xFA6



Hex use case 1 - indicate the 1's and 0's in a byte

Hex use case 2 - 0x10 - represent an amount

000
000

Example 0b01

Address	1000	1001	1002	1003	1004	1005	1006	1007	1008	1009
Value	F6	DC	6D	CD	58	AF	22	49	BC	E3

$\text{char}^* c = 1007$

0x49 0100 1001

↓
73 - 'I'

Example 0b10



What could the value at address 1006 be?

Address	1000	1001	1002	1003	1004	1005	1006	1007
Value	F6	DC	6D	CD	58	AF	22	49

clicker.cs.illinois.edu

Q2

~Code~
340



- A) A bit vector
- B) An ascii character
- C) The middle byte of an int
- D) Part of a struct

Bitwise Operations in C

1. Bit shifting << or >>

2. Bit operations (AND, OR, XOR, NOT)

Bit Shifting

$X = 0100\ 1101$

$X \ll 1$

$1001\ 1010$

0

$X \gg 1$

$0010\ 0110$

* use unsigned

Bit Shifting Example in C

```
2  int main(){
3      unsigned char x = 0x05;
4      x = x << 2;
5      printf("%x\n", x);
6  }
```

$x =$ ~~0000~~ 010100

0001 0100

0x14

What prints?

```
2  int main(){
3      unsigned char x = 0x1E;
4      x = x >> 3;
5      printf("%X\n", x);
6  }
```

16 11 12 13 14 15
A B C D E F
8+4+2

700011110
0000

0000 0011

- A) 0x01
- B) 0xE0
- ☒ C) 0x03
- D) 0x0E

clicker.cs.illinois.edu

Q3

~Code~
340



What prints (challenge)?

```
2  int main(){
3      unsigned int x = 6;
4      x = x << 1;
5      printf("%i\n", x);
6  }
```

→ ~~0...0.01100~~

0...0 1100

= 12
0xC

A) 0x0C

B) 0x12

C) 12

D) 24

clicker.cs.illinois.edu

Q4

~Code~
340



Bit Operations

AND		$\&$
OR		$ $
XOR		$\wedge$
NOT		$\sim$ $\leftarrow$ - in some fonts

$x = 1001$

$y = 0001$

$x \& y$

1001
AND 0001
<u>0001</u>

Bit Operations Example in C

```
int main() {  
    char x = 0x0F;  
    char y = 0x13;  
    char output = x | y;  
    printf("%#x", output);  
}
```

→ 0000 1111
→ 0001 0011
or
0001 1111
→ 0x1F

What prints?

```
int main() {  
    char x = 0x0F;  
    char y = 0x13;  
    char output = x ^ y;  
    printf("%#x", output);  
}
```

→ XOR

x = 0000 1111
y = 0001 0011

0001 1100
1 C

- A) 0x28
- B) 0x1F
- C) 0x30
- D) 0x1C

clicker.cs.illinois.edu

Q5

~Code~
340



What 8 bit value does this produce?

$((\sim 0) \ll 3)$
↑ not ↑

$\sim 0000\ 0000$



- A) 0000 0111
- B) 0000 0011
- ☒ C) 1111 1000
- D) 1111 1011

clicker.cs.illinois.edu

Q6

~Code~
340



What 8 bit value does this produce?

$((\sim 0) \ll 3) \wedge ((\sim 0) \ll 5)$

1111 1000
1110 0000
1110 0000
0001 1000

clicker.cs.illinois.edu

Q7

~Code~
340



- A) 1111 1000
- B) 1110 0000
- C) 0001 1000
- D) 0011 1000

I want just the middle 2 bits from a byte to remain.



mask = 0001 1000

$X \& \text{mask}$

0001 1000

How would I get only the 8 least significant bits from x?

0001
0000
0000

for you don't want the info

1 for keep info

0001
1111
1110

clicker.cs.illinois.edu

Q8

~Code~
340



A

```
int main() {  
    //4 bytes - 8 hex digits  
    int x = 0x1560A0F0;  
    → int mask = 0x000000FF;  
    int output = x & mask;  
    printf("%#x", output);  
}
```

~~B~~

```
int main() {  
    //4 bytes - 8 hex digits  
    int x = 0x1560A0F0;  
    → int mask = 0xFFFFFFFF;  
    int output = x ^ mask;  
    printf("%#x", output);  
}
```

0xF0 → 0x000000F0

Okay... but why?

UTF-8 - MP3.1

Bit sets - MP3.2

UTF-8

Char - 1 byte, can be used to represent ascii

ASCII - mapping from numbers 0-127 to 128 characters

Unicode - bigger ascii numbers $\rightarrow$ ~~char~~ ^{symbols}

$\rightarrow 0x00000000 - 0xFFFFFFFF$ $\leftarrow$ fit in a int

UTF-8 - encoding of unicode code point into varying number of bytes (chars)



UTF-8

Code Point - a number that maps to a symbol

UTF-8 - 1-4 bytes of info that encode a code point

(Encoding) Code point -> UTF-8

1. figure out how many bytes

$0 \times 2 \rightarrow 00 \dots 00 \textcircled{10} - 2 \text{ bits} = 1 \text{ byte}$

$\textcircled{0 \times 108} \rightarrow 0001 \boxed{0000 1000} - 9 \text{ bits} = 2 \text{ bytes}$

bytes

Bits needed	Groups used
$\textcircled{0-7}$	$\textcircled{1}$?
8-11	$\textcircled{2}$
12-16	3 !
$\textcircled{17-21}$	4

(Encoding) Code point -> UTF-8

1 group - 0xxx xxxx

2 group - 110x xxxx 10xx xxxx

3 group - 1110 xxxx 10xx xxxx 10xx xxxx

4 group - 1111 0xxx 10xx xxxx 10xx xxxx 10xx xxxx

Byte	Meaning
0xxxxxxx	only byte of character
10xxxxxx	<u>second, third, or fourth byte of a character</u>
110xxxxx	<u>first byte of a two-byte character</u>
1110xxxx	first byte of a three-byte character
11110xxx	first byte of a four-byte character
11111xxx	invalid

Bits needed	Groups used
0-7	1
8-11	2
12-16	3
17-21	4

(Encoding) Code point -> UTF-8

'a' = 0x61 = 0110 0001 → 7 bits

0110 0001

Byte	Meaning
→ 0xxxxxxx	only byte of character
10xxxxxx	second, third, or fourth byte of a character
→ 110xxxxx	first byte of a two-byte character
1110xxxx	first byte of a three-byte character
11110xxx	first byte of a four-byte character
11111xxx	invalid

Bits needed	Groups used
0-7	1
8-11	2
12-16	3
17-21	4

(Encoding) Code point -> UTF-8



U+0B39 0000 1011 1011 1001 = 12

1110 0000 1010 1110, 1011 1001

0xE0, 0xAE, 0xB9

Byte	Meaning
0xxxxxxx	only byte of character
10xxxxxx	second, third, or fourth byte of a character
110xxxxx	first byte of a two-byte character
<u>1110xxxx</u>	first byte of a three-byte character
11110xxx	first byte of a four-byte character
11111xxx	invalid

Bits needed	Groups used
0-7	1
8-11	2
<u>12-16</u>	<u>3</u>
17-21	4

If I need 9 bits to represent a code point, how many bytes will I need to encode it to UTF-8?

2

clicker.cs.illinois.edu

Q9

~Code~
340



Bits needed	Groups used
0-7	1
8-11	2
12-16	3
17-21	4

How many X slots are there in a 4-group UTF-8 character?

1111 0xxx 10xx xxxx 10xx xxxx

10 xx xxxx

21

clicker.cs.illinois.edu

Q10

~Code~
340



Byte	Meaning
0xxxxxxx	only byte of character
10xxxxxx	second, third, or fourth byte of a character
110xxxxx	first byte of a two-byte character
1110xxxx	first byte of a three-byte character
<u>11110xxx</u>	<u>first byte of a four-byte character</u>
11111xxx	invalid

Bits needed	Groups used
0-7	1
8-11	2
12-16	3
17- <u>21</u>	4

(Decoding) Code point -> UTF-8

- ‡ find start and end
- chop off headers
- put it back together

Byte	Meaning
0xxxxxxx	only byte of character
10xxxxxx	second, third, or fourth byte of a character
110xxxxx	first byte of a two-byte character
1110xxxx	first byte of a three-byte character
11110xxx	first byte of a four-byte character
11111xxx	invalid

Bits needed	Groups used
0–7	1
8–11	2
12–16	3
17–21	4

(Decoding) Code point -> UTF-8

~~0000 1000 1110 0100 1000 0001 1001 0000~~

0x08

0100 0000 0101 0000

= 4

U+4050
0x4050
↓

Byte	Meaning
<u>0xxxxxxx</u>	only byte of character
<u>10xxxxxx</u>	second, third, or fourth byte of a character
110xxxxx	first byte of a two-byte character
1110 xxxx	first byte of a <u>three-byte</u> character
11110xxx	first byte of a four-byte character
11111xxx	invalid

Bits needed	Groups ^{bytes} used
0-7	1
8-11	2
12-16	3
17-21	4

Why UTF-8

- Can encode full unicode range
- Backwards compatible with ASCII
- Space efficient 
- Self-synchronizing 

MP 3.1 - UTF-8

- use bit shifting, operation

code points $\rightarrow$ ~~to~~ UTF-8
(int) (chars)

HW 5

only do
1-3

} - Thursday after
class
MP 3

Bitwise Operations

3.1

Today's LGs - Build on your mental model of how data is stored and interpreted on a computer (1's and 0's).

Be able to converse about bits

Be able to shift bits

Be able to apply logical operations at a bit level

Be able to use bit operations to isolate the bits you need

Be able to convert between code points and UTF-8